

Artificial Intelligence Lab - Lab 08

Travelling on Vacations to Romania

Name: Mehr Ali

Semester: 4th Semester BSCS

Problem Statement

The map of Romania is represented as a graph, with cities as **nodes** and roads as **edges**. We are given a starting city and a goal city. We need to write two functions, one using **BFS** and one using **DFS**, to find a path between the starting city and the goal city.

- **G**: A NetworkX graph object representing the map of Romania
- **start**: The starting city
- **goal**: The goal city

The functions should return a **list of cities** from start to goal, or **None** if no path exists.

Step 1: Import Libraries

```
In [1]: import networkx as nx
import matplotlib.pyplot as plt
```

Step 2: Generate the Romania Graph

```
In [2]: G = nx.Graph()

G.add_edges_from([
    ('Arad', 'Zerind'),
    ('Arad', 'Sibiu'),
    ('Arad', 'Timisoara'),
    ('Zerind', 'Oradea'),
    ('Oradea', 'Sibiu'),
    ('Sibiu', 'Fagaras'),
    ('Sibiu', 'Rimnicu Vilcea'),
    ('Timisoara', 'Lugoj'),
    ('Lugoj', 'Mehadia'),
    ('Mehadia', 'Dobreta'),
    ('Dobreta', 'Craiova'),
    ('Craiova', 'Rimnicu Vilcea'),
    ('Craiova', 'Pitesti'),
```

```
    ('Rimnicu Vilcea', 'Pitesti'),
    ('Fagaras', 'Bucharest'),
    ('Pitesti', 'Bucharest'),
    ('Bucharest', 'Giurgiu'),
    ('Bucharest', 'Urziceni'),
    ('Urziceni', 'Hirsova'),
    ('Urziceni', 'Vaslui'),
    ('Hirsova', 'Eforie'),
    ('Vaslui', 'Iasi'),
    ('Iasi', 'Neamt')
])

print("Graph created!")
print("Cities:", G.number_of_nodes())
print("Roads :", G.number_of_edges())
```

Graph created!

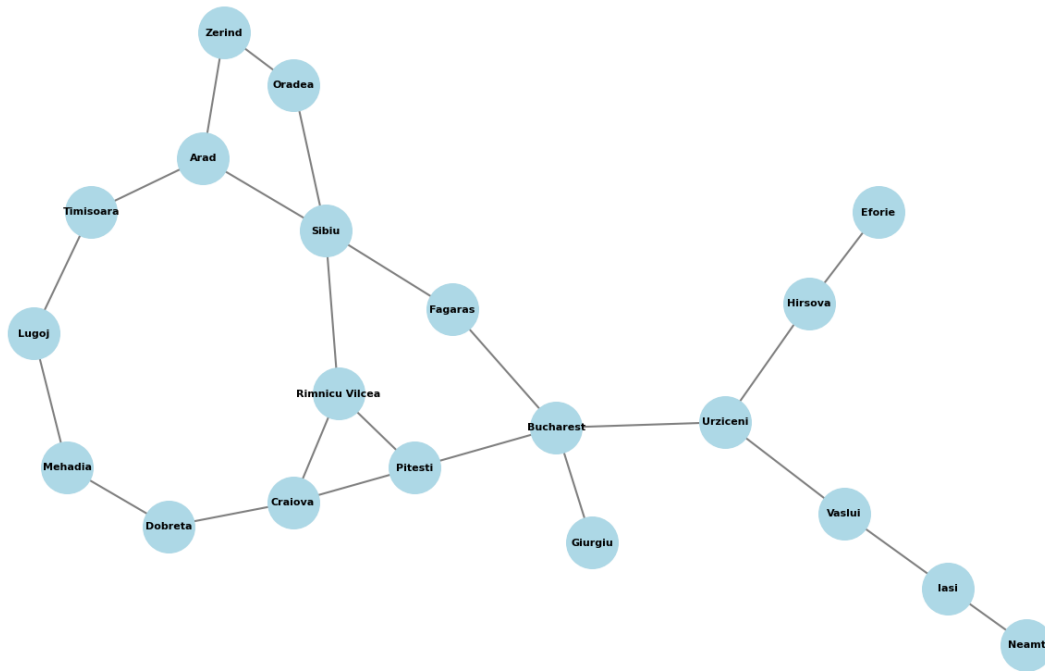
Cities: 20

Roads : 23

Step 3: Draw the Romania Map

```
In [3]: plt.figure(figsize=(13, 8))
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos,
        with_labels=True,
        node_color='lightblue',
        node_size=1600,
        font_size=8,
        font_weight='bold',
        edge_color='gray',
        width=1.5)
plt.title('Romania Map - Cities and Roads', fontsize=14)
plt.show()
```

Romania Map - Cities and Roads



Step 4: BFS Function

BFS uses a **queue**. It explores cities level by level (nearest cities first). It always finds the path with the fewest stops.

```
In [4]: def bfs(G, start, goal):
        queue = [[start]]
        visited = [start]

        while queue:
            path = queue.pop(0)
            city = path[-1]

            for next_city in G.neighbors(city):
                if next_city not in visited:
                    new_path = path + [next_city]
                    if next_city == goal:
                        return new_path
                    visited.append(next_city)
                    queue.append(new_path)

        return None
```

Step 5: DFS Function

DFS uses a **stack**. It explores as deep as possible into one path before backtracking. It finds a valid path but not necessarily the shortest one.

```
In [5]: def dfs(G, start, goal):
        stack = [[start]]
        visited = [start]

        while stack:
            path = stack.pop()
            city = path[-1]

            for next_city in G.neighbors(city):
                if next_city not in visited:
                    new_path = path + [next_city]
                    if next_city == goal:
                        return new_path
                    visited.append(next_city)
                    stack.append(new_path)

        return None
```

Step 6: Test the Functions

```
In [6]: # Test 1 - Arad to Bucharest
        start = 'Arad'
        goal = 'Bucharest'

        print("Start:", start, "| Goal:", goal)
        print("BFS:", bfs(G, start, goal))
        print("DFS:", dfs(G, start, goal))
```

Start: Arad | Goal: Bucharest

BFS: ['Arad', 'Sibiu', 'Fagaras', 'Bucharest']

DFS: ['Arad', 'Timisoara', 'Lugoj', 'Mehadia', 'Dobreta', 'Craiova', 'Pitesti', 'Bucharest']

```
In [7]: # Test 2 - Oradea to Eforie
        start = 'Oradea'
        goal = 'Eforie'

        print("Start:", start, "| Goal:", goal)
        print("BFS:", bfs(G, start, goal))
        print("DFS:", dfs(G, start, goal))
```

Start: Oradea | Goal: Eforie

BFS: ['Oradea', 'Sibiu', 'Fagaras', 'Bucharest', 'Urziceni', 'Hirsova', 'Eforie']

DFS: ['Oradea', 'Sibiu', 'Rimnicu Vilcea', 'Pitesti', 'Bucharest', 'Urziceni', 'Hirsova', 'Eforie']

```
In [8]: # Test 3 - no path exists (returns None)
        G.add_node('Nowhere')
        start = 'Arad'
        goal = 'Nowhere'

        print("Start:", start, "| Goal:", goal)
        print("BFS:", bfs(G, start, goal))
        print("DFS:", dfs(G, start, goal))
```

Start: Arad | Goal: Nowhere

BFS: None

DFS: None

Step 7: Visualize the BFS Path on the Map

```
In [9]: bfs_path = bfs(G, 'Arad', 'Bucharest')
path_edges = [(bfs_path[i], bfs_path[i+1]) for i in range(len(bfs_path)-1)]

plt.figure(figsize=(13, 8))
pos = nx.spring_layout(G, seed=42)

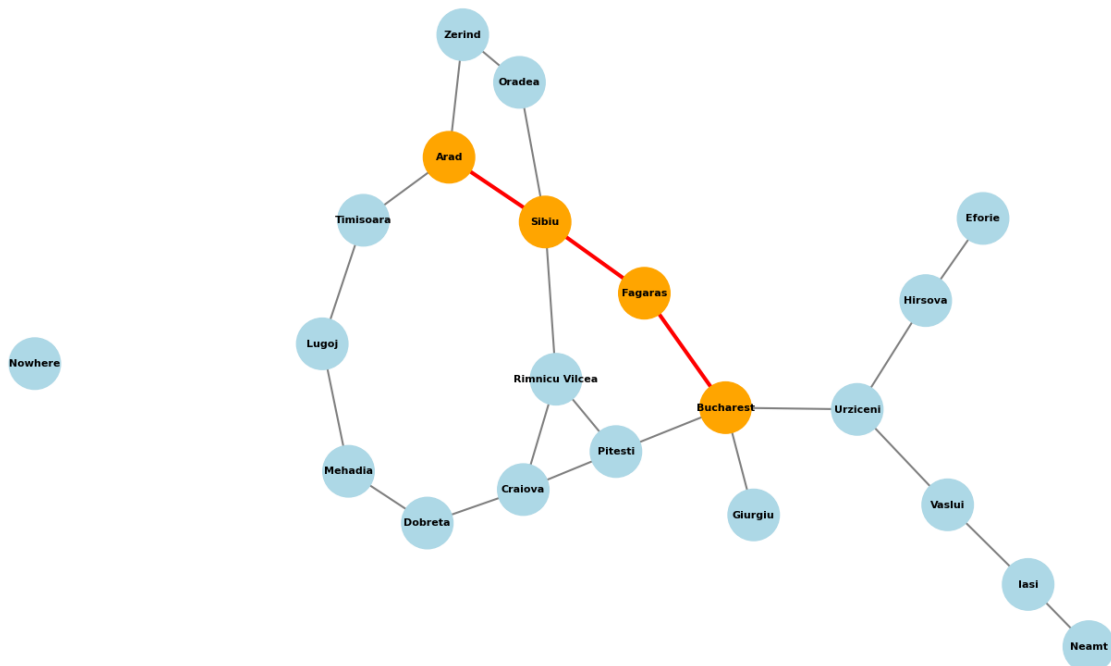
node_colors = ['orange' if n in bfs_path else 'lightblue' for n in G.nodes()]

nx.draw(G, pos,
        with_labels=True,
        node_color=node_colors,
        node_size=1600,
        font_size=8,
        font_weight='bold',
        edge_color='gray',
        width=1.5)

nx.draw_networkx_edges(G, pos, edgelist=path_edges, edge_color='red', width=3)

plt.title('BFS Path: Arad to Bucharest (orange = path cities, red = path roads)',
plt.show()
```

BFS Path: Arad to Bucharest (orange = path cities, red = path roads)



Short Explanation

	BFS	DFS
Data Structure	Queue — <code>pop(0)</code>	Stack — <code>pop()</code>
Explores	Level by level	Deep first
Shortest path?	Yes	Not always
Returns	List of cities or None	List of cities or None

The only real difference between BFS and DFS in the code is `pop(0)` vs `pop()`. Everything else is the same.